

Flatland Revisited: The Effects of Non-Planar Floors on Large-Scale Warehouse Deployment of Autonomous Mobile Robots

Jaimie Carlson¹

Abstract—An actively running warehouse is often considered a relatively safe environment for robot deployment, as compared to more exotic fields such as outdoor, underwater, or aerospace locations. However, even in such a controlled environment, when deploying many robots at scale over extended periods of time, relatively minor environmental imperfections can lead to numerous system failures. This paper discusses the common assumption that indoor mobile robots are operating on a flat, level floor. Even with relatively flat floors, non-flat areas can cause problems for localization and perception. This paper presents a case study of the impact of an improperly applied flat floor assumption, which caused inaccuracies in 2D pose estimation of a floor fiducial from a single RGB image point. To solve this problem, a method to estimate pose of a fiducial on the floor within known flatness bounds from two or three image points was developed. Using this method for a deployment at scale in a testing site with non-flat floor eliminated pose estimation failures, removing over one hundred task failures over three months.

I. INTRODUCTION

In comparison to field deployments in the disciplines of outdoor, aerospace, or underwater robotics, warehouse robot deployments take place in an environment which is relatively robot-friendly and well-understood. First, the environment is assumed to be inside a building designed for humans, meaning it has shelter from weather, good lighting, and relatively flat floors over which a ground robot can navigate. Second, the environment is assumed to be well-mapped. A robot is deployed into a building with a known blueprint of walls and floors, and often a construction specification defining that blueprint’s accuracy and precision.

Because these assumptions are typically reliable, the software of a warehouse robot may grow to explicitly depend on them. However, warehouse robots are often deployed with many robots per site, operating for very long durations. A larger-scale deployment increases the chance that a robot encounters an edge case or out-of-distribution event where one of these assumptions is not met, potentially causing problems in field deployment.

Here, we specifically examine the assumption that a robot is operating on a flat and level floor. Using this assumption is helpful: it allows a robot to represent its state in $\text{SE}(2)$ space instead of $\text{SE}(3)$ space, and rely primarily on two-dimensional perception and mapping techniques. This simplifies state representation and saves on compute, which can be important to reduce cost. However, as a warehouse floor specification may define, this flat-floor assumption is not

always accurate, as illustrated in Figure 1. Even when floors adhere to reasonable construction specifications, small errors of floor non-flatness can easily cause large distance errors in pose estimation techniques that assume a two-dimensional world.

This paper makes the following contributions:

- Presenting a literature review examining the effects of non-flat floor on ground robots deployed to a field.
- Placing an upper bound on image feature pose estimation error caused by floor non-flatness.
- Presenting a case study where the erroneous assumption of floor flatness caused pose estimation failures across a large-scale warehouse robotic field deployment.
- Demonstrating a solution to that case study, and quantifying its effect on task success rates.

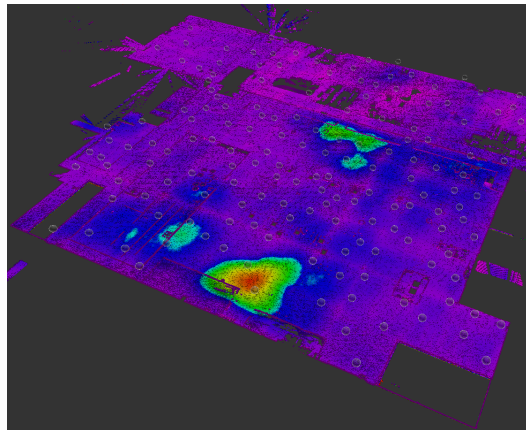


Fig. 1. Survey of a non-flat test floor; height of points with respect to construction specification varies from -11 cm (red) to +4 cm (purple).

II. FLOOR SPECIFICATIONS

Lack of flatness can occur in warehouse floors due to joints across different slabs of concrete or roughness in floor material. A warehouse floor specification stipulates the maximum deviations from perfect floor flatness and levelness allowed for a given floor. As defined by the ASTM standard E1155, *floor flatness* F_F is a dimensionless number measuring floor planarity, and *floor levelness* F_L is a dimensionless number measuring floor perpendicularity to the direction of gravity as measured by a bubble level [1].

Note that it isn’t practical for surveyors to measure flatness and levelness along every possible point in a floor, so lines are picked along which to sample these measurements [1]. Thus, it is possible for F_F to exceed any bounds in a local

¹Jaimie Carlson is with Amazon Global Robotics carjaimi@amazon.com

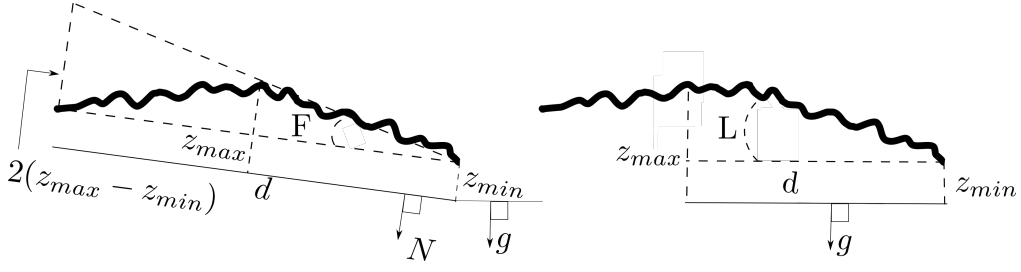


Fig. 2. **(Left)** Flatness bound $F = ||\text{atan}(\frac{2(z_{\max}-z_{\min})}{d})||$. **(Right)** Levelness bound $L = ||\text{atan}(\frac{z_{\max}-z_{\min}}{d})||$

area which was not sampled in a line measurement. Also note that over the long lifespan of a robotic deployment, the use of robots itself, as well as other vehicles like forklifts, can wear down the floor and cause it to deviate from older specifications, as occurred in the test site shown in Figure 1.

For this paper, we use definitions for flatness and levelness bounds in units of angles, which are easier to reason about than the unitless F_F and F_L . As illustrated in Figure 2 (Left), flatness bound F is defined as the maximum angle deviation over a standard distance d , where heights $z_{\min}$ and $z_{\max}$ are defined with respect to the vector to the normal plane N over distance d , not with respect to the gravity normal vector g . As illustrated in Figure 2 (Right), levelness bound L is defined as the total angle deviation with respect to gravity, where heights $z_{\min}$ and $z_{\max}$ are measured over a given standard distance d measured perpendicular to g [2].

III. LITERATURE REVIEW

A. Motion Planning

Most literature about motion planning for ground robots in non-planar environments focuses on significantly non-flat environments like mountainous off-road terrain, ramps, or parking garages. Often, these techniques take a mesh or point cloud of the environment and convert it into a cost map with scores based on traversability, with a high cost for obstacles, cliffs, or uneven surfaces where a ground robot cannot safely stand [3], [4], [5]. However, for warehouses, deviations from floor flatness are rarely high enough to require such techniques. A robot can generally stably stand on every point on the ground - there is much more concern on how non-flatness affects the accuracy of perception.

B. Ranged Perception

Registration between range-and-bearing measurements, such as time-of-flight distance scans or point clouds produced by a 2D LIDAR, can fail drastically on non-planar floors. Figure 3 (Top) shows one such case: initially, a robot's range sensor measures distance ℓ_1 to an object. After moving forward distance ϵ , it expects to measure $\ell_2 = \ell_1 - \epsilon$ to the same object. However, if this movement places the robot on a section of floor pitched upward at angle F , the actual measured distance becomes $\ell_2 = \frac{\ell_1 - \epsilon}{\cos(F)}$. The discrepancy between expected and actual ℓ_2 distance, $(\ell_1 - \epsilon)(\frac{1}{\cos(F)} - 1)$, is fairly small for most reasonable values of F . However, much greater differences between expected and actual ℓ_2

distance can occur if the floor pitches downwards at angle F between robot and object. This causes the sensor to perceive the floor instead of the object, as shown in Figure 3 (Bottom).

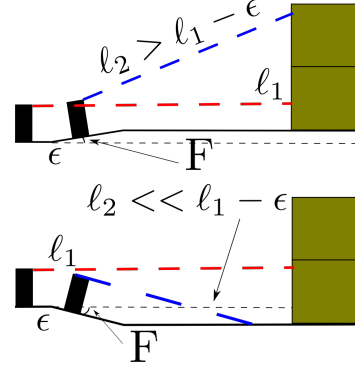


Fig. 3. **(Top)** For a floor pitched upwards, $\ell_2 \approx \ell_1 - \epsilon$. **(Bottom)** For a floor pitched downwards, $\ell_2 \ll \ell_1 - \epsilon$, as the range sensor measures distance to the floor instead of an expected object.

Even greater distance errors between expected and actual ℓ_2 can occur if the sensor is viewing an object with a non-horizontally consistent cross section, as shown in Figure 4 (Top). Tilting the sensor causes it to view a completely different part of the object. This creates ambiguity; an observer comparing ℓ_1 and ℓ_2 cannot tell whether the robot has moved, the object has moved, or the robot has moved to an area of floor with nonzero pitch F .

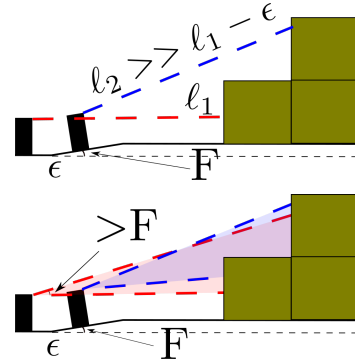


Fig. 4. **(Top)** For an object with a non-horizontally consistent cross section, $\ell_2 \gg \ell_1 - \epsilon$. **(Bottom)** A 3D scan with vertical FOV $> F$ will have overlap allowing for 6-DOF registration.

This issue is a particular problem with 2D scans. However,

the use of 3D scans with a wide ($> F$) vertical field of view provides a solution. Scans taken at pitches 0 and F should still have some overlap, allowing a registration method like ICP to find an $SE(3)$ transformation between them, as shown in Figure 4 (Bottom).

Floor non-flatness can affect the accuracy of registration between ranged distance measurements - a critical upstream component of SLAM systems that heavily rely on LIDAR. This problem is not extensively discussed in the literature of LIDAR-odometry SLAM systems, as many ground robots with more compute available simply use a full 6-DoF mapping system to map in $SE(3)$, and avoid making a flatness assumption entirely [6], [7], [8]. Others, like Gmapping, map in $SE(2)$, which works well in relatively small or flat environments [9]. One localization system which explicitly addressed this issue was developed as a result of the LANDroids DARPA Challenge, which required robots to navigate indoor environments with imprecise CAD blueprints and uneven floors. This necessitated the creation of a scan matching technique that was robust against uneven floors but was limited to environments with perpendicular walls [10]. To deal with floor non-levelness, other 2D SLAM systems such as Hector-SLAM, Cartographer SLAM, LOAM, and DLIOM use an IMU to project 2D LIDAR scans into a level ground plane [11], [12], [13], [14]. Another notable approach first has a mapping phase to generate a point cloud of the non-flat environment using a full 6-DoF SLAM system and convert it into an Octomap and gridmap. At runtime, robots perform AMCL for localization in $SE(2)$ against that Octomap, allowing for an accurate representation of floor non-flatness, but less compute used during deployment [15].

C. Image Perception

Because they provide only bearing and not range data, perception techniques using camera data to see objects on a floor are more susceptible to errors caused by nonflat floors. For instance, the bird's-eye-view (BEV) paradigm used in image segmentation and pose estimation heavily relies on a top-down planar view, created with either 2D-3D methods, or 3D-2D methods [16]. 3D-2D methods explicitly project 2D features onto a 3D ground plane with $z = 0$, meaning that they can pass non-flatness errors onto pose estimates [17]. However, 2D-3D methods, such as "Lift, Splat, Shoot," can be more robust in the case of erroneous flat floor assumptions, because they "lift" 2D features into a possible distribution of 3D points over the projected ray of the image feature, not just one 3D point with height $z = 0$ [18].

Other perception techniques avoid relying on projection to a ground plane by leveraging known properties of objects on the floor. For instance, the use of visual fiducials, such as April Tags [19], ARTags [20], or ArUco Tags [21], involves detection of the corners of a quadrilateral in an RGB image. Then, Perspective-n-Points (PnP) can be applied between those corners in image space and the corners of a tag with known size in world space, to get a pose estimation of the fiducial with respect to the camera. PnP requires at least three points to extract four possible solutions, and at least four

points to extract an unambiguous solution [22], [23]. When detecting a visual fiducial on the ground, tag pose estimation may return a pose with nonzero z -height. Projecting this pose from $SE(3)$ to $SE(2)$ to use it in an $SE(2)$ system may simply involve setting its z -height to 0. In this case, perceiving a visual fiducial on the floor does not directly require the assumption that z -height of the object is zero, since extra information about the fiducial, like its size, can be used to get a robust pose estimate.

IV. CASE STUDY: RGB POSE ESTIMATION: METHOD



Fig. 5. (Top) A whole fiducial. (Bottom) A ripped fiducial, for which all corners cannot be detected.

This case study focuses on the effects of non-flat floor on a fleet of autonomous robots which navigate through warehouses. They are deployed to a warehouse testing area, with tens of robots deployed in $\approx 1,000 m^2$ of floor area.

In this environment, each robot estimates the pose of a target fiducial on the floor through an RGB camera image, in order to drive onto that fiducial to complete a task. This fiducial is a large blue square. One might expect that the corners of this fiducial could be detected and used to estimate pose through PnP, through a similar process as that described for April or ArUco tags in section III.C. However, these fiducials are typically placed in a busy environment, where their corners can be easily torn, as shown in Figure 5. As a direct consequence of the dynamic field environment to which the robot is deployed, it cannot be assumed that all four corners of the fiducial can be extracted individually. Instead, semantic segmentation is used to label all pixels in the image belonging to the fiducial, as detailed more fully in [24]. The centroid of the pixels labeled with semantic class "fiducial" is treated as a 2D image feature, providing some information about the fiducial position.

A. Method for Flat-Floor Pose Estimation

Since not all four points can be extracted, the assumption of a flat floor can be used to recover scale from a single image feature point. The projection equation projects a pixel $[u, v]$ in the image space of a camera with intrinsics K and camera pose in the robot frame cT_r to a ray of poses $\frac{1}{\lambda} K {}^cT_r^T p = \frac{1}{\lambda} K {}^cT_r^T \begin{bmatrix} rX & rY & rZ & 1 \end{bmatrix}^T$, with scale factor λ . To solve for a single point along that ray which gives the fiducial's

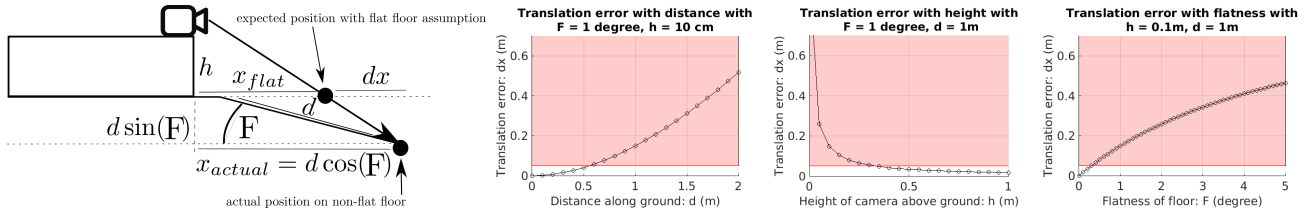


Fig. 6. A camera mounted at height h , seeing an object at distance d away along a floor with flatness bound F , can have up to translation error dx in its estimation of the object's pose. dx increases with d and F , and decreases with h . Translation errors above 5 cm (red) cause task failures.

3D position, assume that the fiducial and robot are on the ground, so ${}^rZ = 0$:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K^c T_r \begin{bmatrix} {}^rX \\ {}^rY \\ 0 \\ 1 \end{bmatrix} \quad (1)$$

There are 3 unknowns (λ , rX , and rY), so that an individual point along the ray can be solved for with 3 linear equations. The full solution, presented in the Appendix, is used to convert the centroid of fiducial segmentation pixels $[u, v]$ to the position of the center of the fiducial (${}^rX, {}^rY$).

B. Error in Flat-Floor Pose Estimation Method

In the worst-case scenario for pose estimation, a robot with a camera positioned at height h is at the edge of a floor plane with maximum flatness angle error F , trying to perceive a visual feature at distance d away, and project it to $({}^rX, {}^rY)$, as shown in Figure 6. Increased distance from the camera d , lower camera mounting height h , or increased floor flatness bound F increase the maximum translation error in feature perception caused by the flat floor assumption, dx .

$$dx = x_{actual} - x_{flat} = d \cos(F) \left(1 - \frac{h}{h + d \sin(F)} \right) \quad (2)$$

In this use case, a camera mounted at 10 cm h above the ground needs to detect floor features from d of 1 to 2 m away, on a floor with flatness bounds F from 1 to 5 degrees, with an accuracy of $dx < 5$ cm. As expected based on Figure 6 and shown in Figure 8, the pose estimation of fiducials sometimes failed to meet this requirement. Good segmentation meant that the fiducial centroid was detected accurately, but comparing the reprojected expected locations of the fiducial corners to their measured positions in the image showed that depth λ was underestimated and dx was too high.

When considering ways to reduce dx , decreasing floor flatness angle bounds F by smoothing a floor is often cost-prohibitive. Increasing camera mounting height h can entail major hardware changes, which are expensive and sometimes incompatible with robot functionality. For instance, this particular robot has to be small enough to drive underneath some obstacles, so it cannot mount its camera 1m above the ground. Detecting features continuously as the robot approaches a fiducial can help lower d to some extent. However, if d is too low, a feature will leave the camera's field of view entirely.

Another option is to eliminate the flat floor assumption for pose estimation entirely, by providing a depth estimate of the point to compute λ . This could be done by adding additional depth sensors, which are expensive, and may not have the needed resolution to provide depth at a specific feature pixel. More cheaply, multiple images from the same camera could be used to create a virtual stereo pair, then perform stereo block matching. However, stereo block matching may fail in textureless regions, like indoor floors. Thus, this study aimed to find a pose estimation method which relies on RGB alone and is robust to the degree of floor non-flatness expected in a warehouse environment.

C. RGB-Only Non-Flat-Floor Pose Estimation Method

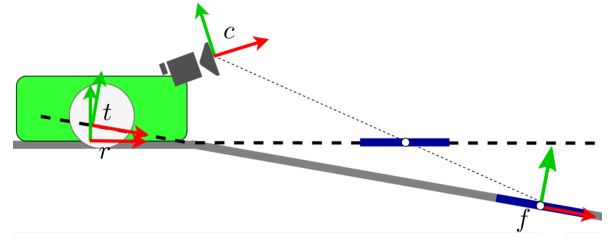


Fig. 7. Camera frame c , robot frame r , fiducial frame f , and frame of floor tangent to fiducial t .

A method to perform fiducial pose estimation without the flat floor assumption was developed with two key findings. First, the method must work within the worst-case scenario for the floor's flatness bounds, as illustrated in Figure 7. Second, more feature points must be provided. As previously noted, all four corners of a fiducial cannot reliably be detected. However, image segmentation can usually be processed to identify the centroid and at least one of the two processed corners, even if a fiducial is torn.

First, these relevant points are extracted from the segmentation. Given an image, a semantic segmentation is run to label fiducial pixels [24]. The centroid is then computed from that image segmentation. The largest connected component of all fiducial-labeled pixels is found, and its external contours are defined by OpenCV's `findContours()` function. Each point $[u, v]$ on the contour is transformed from image frame $\mathcal{I}$ to ground space using the projection equation from Section IV.A. The centroid of the points in ground space is calculated as $({}^f x_t, {}^f y_t)$, and projected back to image space as ${}^{\mathcal{I}} \hat{p}_{center}$. Once the centroid is known, the bottom corners of the fiducial can be computed from the image segmentation. The rotated

bounding box that encloses the segmentation contours is generated by OpenCV's `minAreaRect()` function - this is shown as the pink box in Figure 8. Of the contour points in the bottom half of the image that are below a distance threshold ϵ from the rotated bounding box, the point closest to the left bounding box corner is chosen as the bottom-left fiducial corner ${}^{\mathcal{I}}\hat{p}_{left} \in \mathbf{R}^2$, and the point closest to the right bounding box corner is chosen as the bottom-right fiducial corner ${}^{\mathcal{I}}\hat{p}_{right} \in \mathbf{R}^2$.

Second, those points are used to compute the pose of the tangent plane with respect to the fiducial, ${}^fT_t \in \mathbf{SE}(2)$. The centroid $({}^fx_t, {}^fy_t)$ is used for the position of fT_t , while the corner points are used to calculate the angle of fT_t . ${}^{\mathcal{I}}\hat{p}_{left}$ is projected to ground space using the projection equation solution from Section IV.A as $({}^{left}x_t, {}^{left}y_t)$, in order to calculate the angle of the fiducial ${}^f\theta_t$:

$${}^f\theta_t = -\frac{\pi}{4} + \text{atan}\left(\frac{{}^fx_t - {}^{left}x_t}{{}^fy_t - {}^{left}y_t}\right) \quad (3)$$

So, the pose of the fiducial f with respect to tangent floor plane t can be written as a transformation:

$${}^fT_t = \begin{bmatrix} \cos({}^f\theta_t) & -\sin({}^f\theta_t) & 0 & {}^fx_t \\ \sin({}^f\theta_t) & \cos({}^f\theta_t) & 0 & {}^fy_t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

Third, define the pose of the robot r with respect to tangent floor plane t as a transformation matrix. ${}^r\beta_t$ is the pitch rotation from t to r , and rz_t is translation in z-direction from t to r . ${}^r\beta_t$ and rz_t are unknown, but within the limits of floor flatness bounds. Assume no roll or yaw between r and t .

$${}^rT_t = \begin{bmatrix} \cos({}^r\beta_t) & 0 & \sin({}^r\beta_t) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin({}^r\beta_t) & 0 & \cos({}^r\beta_t) & {}^rz_t \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

Finally, an optimization problem is initialized to find the pose of the fiducial in the robot frame which minimizes the reprojection error between detected image points and projected expected image points. The centroid, bottom-left, and bottom-right corners of the fiducial with side length L in frame f are:

$$\begin{aligned} {}^fp_{center} &= [0 \ 0 \ 0 \ 1]^T \\ {}^fp_{left} &= [-L/2 \ L/2 \ 0 \ 1]^T \\ {}^fp_{right} &= [-L/2 \ -L/2 \ 0 \ 1]^T \end{aligned}$$

A point in the fiducial frame, ${}^fp \in \mathbf{R}^3$, can be projected onto the image frame $\mathcal{I}$ to expected image point ${}^{\mathcal{I}}p \in \mathbf{R}^2$. Here, ${}^{\mathcal{I}}T_f = {}^fT_t^{-1}$:

$$\lambda {}^{\mathcal{I}}p = K \times {}^cT_r \times {}^rT_t \times {}^tT_f \times {}^fp \quad (6)$$

For all $pts = \{center, left, right\}$, the pose rT_f is found which minimizes the reprojection error between measured point ${}^{\mathcal{I}}\hat{p}_i$, and expected point ${}^{\mathcal{I}}p_i$ projected from fp_i :

$$\arg \min_{{}^rT_t \times {}^tT_f \in \text{Bounds}({}^r\hat{T}_f)} = \sum_{i \in pts} \|{}^{\mathcal{I}}\hat{p}_i - {}^{\mathcal{I}}p_i\| \quad (7)$$

$$\arg \min_{{}^rT_t \times {}^tT_f \in \text{Bounds}({}^r\hat{T}_f)} = \sum_{i \in pts} \left\| {}^{\mathcal{I}}\hat{p}_i - \frac{1}{\lambda} K \times {}^cT_r \times {}^rT_t \times {}^tT_f \times {}^fp_i \right\| \quad (8)$$

In the optimization problem, the variables are $({}^fx_t, {}^fy_t, {}^f\theta_t, {}^r\beta_t, {}^rz_t)$. They are allowed to vary within limits that keep ${}^rT_t \times {}^tT_f$ within $\text{Bounds}({}^r\hat{T}_f)$. These limits are centered around the initial assumption that the floor is flat, so ${}^rT_t = I$, and ${}^r\hat{T}_f = {}^rT_t \times {}^tT_f = {}^tT_f$. Around this starting pose, the pitch ${}^r\beta_t$ can vary based on the floor flatness bounds $\pm F$, and rz_t can vary $\pm d \sin(F)$ for expected detection distance d . Figure 8 shows the improvement this optimization achieves with reprojected points in blue and the actual segmentation, since its corners were used in the optimization.

This method has some robustness to poor segmentations. If both lower corners are detected, $pts = \{center, left, right\}$. If only one lower corner is detected, $i \in pts = \{center, left\}$ or $i \in pts = \{center, right\}$. As shown in Figure 7 (Bottom left), this requires fewer points to be detected, making the method more robust to fiducial damage on a busy warehouse floor and allowing it to produce an estimation where PnP would fail. In the case of an extremely poor segmentation, no corner points can be identified accurately enough to segment out the fiducial. Segmentation area percentage p is calculated as $\frac{A}{L^2}$, where A is the area of the segmentation contour points projected to the ground plane. If p is smaller than an experimentally set threshold, assume the segmentation was poor, and fall back to the flat floor assumption.

Figure 8 shows the improvement this optimization achieves with reprojected points in blue and the actual segmentation, since its corners were used in the optimization. In conclusion, inserting slack variables to explicitly estimate non-zero floor pitch and height, and adding more points to provide more information so that this problem is not underdetermined, helps solve the problem of inaccurate pose estimation through image features on non-flat floors.

V. CASE STUDY: RGB POSE ESTIMATION: RESULTS

A. Controlled Experiments

During normal operation at the test site, tasks are allocated for the robot to detect different fiducials. Figure 9 summarizes 4 different medium-scale tests that were run at the testing site shown in Figure 1, across the course of 3 months. The "Flat" category aggregates two trials run with the flat-floor-assuming pose estimation described in IV.A; and the "Nonflat" category aggregates two trials run with three-point pose estimation that does not assume a flat floor, described in IV.C. Each individual trial included 10 robots, running for 4 to 5 hours, and about 300 pose estimation tasks. Tasks that failed for unrelated reasons were removed from the trial (n_{fail_other} in Figure 9).

During the "Flat" tests, it was anecdotally noted that the flat-floor task failures were repeated with different robots at the same fiducials, located in places where the floor was sloping downwards more, based on the survey shown in Figure 1. The percentage of all trials that failed due to flat

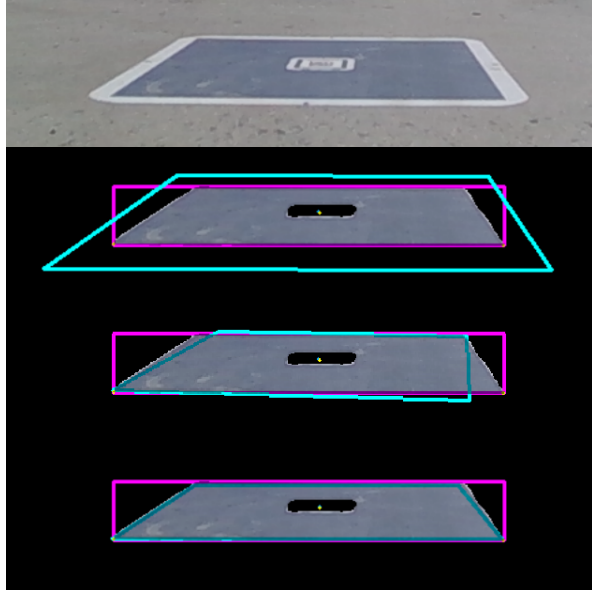


Fig. 8. (Top left) Input image: The fiducial, a blue square, is a target for the robot to drive onto.

(Top right) Raw projection: The square is segmented accurately from the image. Reprojecting the assumed poses of the corners of the fiducial with the flat-floor assumption (blue quadrilateral) onto the actual segmentation (pink rectangle) illustrates that depth of the feature is underestimated. ${}^t x_f = 1.01m$, ${}^t y_f = 0.03m$, ${}^t \theta_f = -0.01$ rad, ${}^r z_t = 0.000m$, ${}^r \beta_t = 0.00$ rad.

(Bottom left) Two-point solution: Fitting centroid and lower left corner better estimates fiducial's depth. ${}^t x_f = 1.18m$, ${}^t y_f = 0.04m$, ${}^t \theta_f = -0.10$ rad, ${}^r z_t = -0.004m$, ${}^r \beta_t = 0.03$ rad.

(Bottom right) Three-point solution: Fitting centroid and both lower corners better estimates fiducial depth and angle. ${}^t x_f = 1.14m$, ${}^t y_f = 0.04m$, ${}^t \theta_f = 0.01$ rad, ${}^r z_t = 0.040m$, ${}^r \beta_t = 0.06$ rad.

floor assumptions fell from 0.95% to 0.00% when the flat floor assumption was removed.

Trial	Flat	Nonflat
n_{total}	645	584
n_{fail_other}	10	12
n	635	572
n_{fail_flat}	6	0
$p_{fail} = (n_{fail_flat} + n_{other})/n_{total}$	2.48%	2.06%
$p_{fail_flat_total} = n_{fail_flat}/n_{total}$	0.93%	0.00%
$p_{fail_flat} = n_{fail_flat}/n$	0.95%	0.00%

Fig. 9. Number of fiducial detection attempts n_{total} , number of fiducial detection attempts n discounting the n_{fail_other} tasks that failed for reasons unrelated to fiducial detection, and number of failures directly caused by flat floor assumption n_{fail_flat} .

B. Long-Term Observation

In a separate long-term study, three months of data were collected at the test site, with 30 unique robots using the non-flat-floor-assuming method from IV.C. The study recorded $n_{total} = 11318$ fiducial detection attempts. In the test, $n_{fail_flat} + n_{other} = 108$ failures were seen, for an overall failure rate p_{fail} of 0.95%, which is markedly lower than the baseline p_{fail} rate of 2.48% observed in the controlled

experiments. From the Flat trials, $p_{fail_flat_total}$ of 0.93% indicates that if the flat floor assumption was made in the long-term trial, it could have caused up to 105 failures, which did not occur here. Over several months, a change which affects less than 1% of all trial outcomes can prevent hundreds of task failures.

The variation in z - and pitch estimation during these three months of tasks can be seen in Figure 10, where the returned values (± 12 cm z , ± 0.06 rad pitch) roughly match survey bounds for the site. Fiducial detections have a greater range across robots than across fiducial locations, indicating that floor tilt, not individual robot camera calibration, dominates error values. However, variation across fiducial locations is still nontrivial, indicating that different robots estimate floor pitch differently when arriving at the same fiducial. This could be because robots arrive at fiducials from slightly different approach positions with different floor pitches and z -heights, or because extrinsic calibration error in ${}^c T_r$ affects estimated floor pitch and z for a given robot.

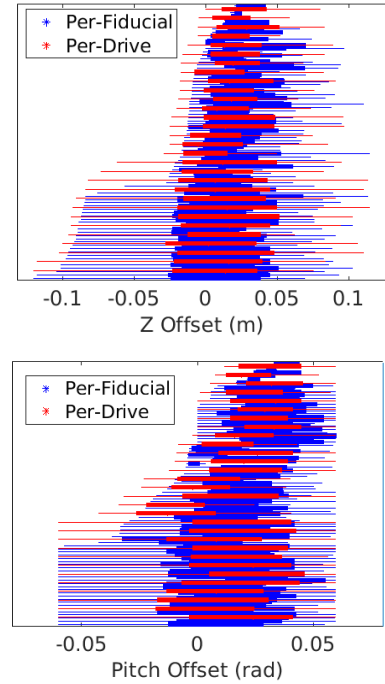


Fig. 10. Boxplots per-horizontal-line show z ${}^r z_t$ and pitch ${}^r \beta_t$ per fiducial or per drive robot. Fiducial detections have a lower range for z - and pitch per fiducial location (average z -range 0.106m; average pitch-range 0.075rad) than per-robot (average z -range 0.135m; average pitch-range 0.086rad). This indicates that variation in calculated z - and pitch-values is not primarily due to per-robot calibration errors, but due to per-site floor flatness.

VI. CONCLUSION

Focusing specifically on the imperfection of non-planar floors, this paper presents a case study where pose estimations of a fiducial on a non-flat floor using a flat floor assumption failed. This was addressed via detecting one or two additional points on the floor fiducial, and putting them in an optimization problem to minimize reprojection error while assuming a pose within known flatness bounds F . This method eliminates pose estimation errors that would have otherwise caused

hundreds of task failures, while still using fewer points than PnP. Though this case study addresses a problem very specific to our case study, the general principle remains that even a well-characterized environment for deployment will have imperfections. For a successful field deployment at a large scale, it is key to understand those flaws and compensate for them robustly.

REFERENCES

- [1] ASTM. Standard test method for determining ff floor flatness and fl floor levelness numbers, 4 2024.
- [2] Amazon Robotics Hardware Engineering. Environmental specification, proteus drive unit, 2024.
- [3] Philipp Krüsi, Paul Furgale, Michael Bosse, and Roland Siegwart. Driving on point clouds: Motion planning, trajectory optimization, and terrain assessment in generic nonplanar environments. *Journal of Field Robotics*, 34, 2017.
- [4] Ming Liu. Robotic online path planning on point cloud. *IEEE Transactions on Cybernetics*, 46:1217–1228, 5 2016.
- [5] Jingping Wang, Long Xu, Haoran Fu, Zehui Meng, Chao Xu, Yanjun Cao, Ximin Lyu, and Fei Gao. Towards efficient trajectory generation for ground robots beyond 2d environment. 2 2023.
- [6] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, Jose Neira, Ian Reid, and John J. Leonard. Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. *IEEE Transactions on Robotics*, 32, 2016.
- [7] Raul Mur-Artal, J. M. M. Montiel, and Juan D. Tardos. Orb-slam: A versatile and accurate monocular slam system. *IEEE Transactions on Robotics*, 31(5):1147–1163, October 2015.
- [8] Antoni Rosinol, Andrew Violette, Marcus Abate, Nathan Hughes, Yun Chang, Jingnan Shi, Arjun Gupta, and Luca Carlone. Kimera: From slam to spatial perception with 3d dynamic scene graphs. *International Journal of Robotics Research*, 40, 2021.
- [9] Giorgio Grisetti, Cyrill Stachniss, and Wolfram Burgard. Improved techniques for grid mapping with rao-blackwellized particle filters. *IEEE Transactions on Robotics*, 23(1):34–46, 2007.
- [10] Peter Abeles. Robust local localization for indoor environments with uneven floors and inaccurate maps. In *Proceedings - 2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2011.
- [11] S. Kohlbrecher, J. Meyer, O. von Stryk, and U. Klingauf. A flexible and scalable slam system with full 3d motion estimation. In *Proc. IEEE International Symposium on Safety, Security and Rescue Robotics (SSRR)*. IEEE, November 2011.
- [12] Wolfgang Hess, Damon Kohler, Holger Rapp, and Daniel Andor. Real-time loop closure in 2d lidar slam. In *Proceedings - IEEE International Conference on Robotics and Automation*, volume 2016-June, 2016.
- [13] Ji Zhang and Sanjiv Singh. Loam: Lidar odometry and mapping in real-time. In Dieter Fox, Lydia E. Kavraki, and Hanna Kurniawati, editors, *Robotics: Science and Systems*, 2014.
- [14] Kenny Chen, Ryan Nemiroff, and Brett T. Lopez. Direct lidar-inertial odometry and mapping: Perceptive and connective slam, 2023.
- [15] Francisco Martín Rico, José Miguel Guerrero Hernández, Rodrigo Pérez Rodríguez, Juan Diego Peña Narváez, and Alberto García Gómez-Jacinto. Open source robot localization for non-planar environments. 9 2023.
- [16] Hongyang Li, Chonghao Sima, Jifeng Dai, Wenhai Wang, Lewei Lu, Huijie Wang, Jia Zeng, Zhiqi Li, Jiazhi Yang, Hanming Deng, Hao Tian, Enze Xie, Jiangwei Xie, Li Chen, Tianyu Li, Yang Li, Yulu Gao, Xiaosong Jia, Si Liu, Jianping Shi, Dahua Lin, and Yu Qiao. Delving into the devils of bird's-eye-view perception: A review, evaluation and recipe. 9 2022.
- [17] Yunzhong Hou, Liang Zheng, and Stephen Gould. Multiview detection with feature perspective transformation. 7 2020.
- [18] Jonah Philion and Sanja Fidler. Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d. 8 2020.
- [19] Edwin Olson. Apriltag: A robust and flexible visual fiducial system. In *Proceedings - IEEE International Conference on Robotics and Automation*, 2011.
- [20] Mark Fiala. Artag, a fiducial marker system using digital techniques. In *Proceedings - 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, CVPR 2005*, volume II, 2005.

- [21] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez. Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47, 2014.
- [22] Xiao Shan Gao, Xiao Rong Hou, Jianliang Tang, and Hang Fei Cheng. Complete solution classification for the perspective-three-point problem. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 25, 2003.
- [23] Vincent Lepetit, Francesc Moreno-Noguer, and Pascal Fua. Epnnp: An accurate o(n) solution to the pnp problem. *International Journal of Computer Vision*, 81, 2009.
- [24] Xia Li, Jaimie Carlson, Xiangxin Kong, Lionel Gueguen, John Enright, Sadat Shaik, Carl Delsey, Sophie Li, Ben Knorlein, and Andreas Kolling. Autonomous mobility in fulfillment centers - precision storage cell segmentation. *Amazon Machine Learning Conference*, 2023.

ACKNOWLEDGEMENTS

Thanks to Andreas Kolling, Chris Keane, and Pablo Ortiz for comments and feedback on this paper. Thanks to Xia Li for paper feedback and many helpful discussions on fiducial segmentation.

APPENDIX

Flat Floor Estimation

Assume that $H = K \times^c T_r$ can be found from camera intrinsics and extrinsics. Refer to its elements as h_{ij} with indices i and j .

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} & h_{02} & h_{03} \\ h_{10} & h_{11} & h_{12} & h_{13} \\ h_{20} & h_{21} & h_{22} & h_{23} \end{bmatrix} \begin{bmatrix} {}^r X \\ {}^r Y \\ 0 \\ 1 \end{bmatrix} \quad (9)$$

Known: h_{ij} elements, u, v . Unknown: $\lambda, {}^r X, {}^r Y$.

Provide one equation for each row of the matrix equation:

$$\lambda u = h_{00}^r X + h_{01}^r Y + h_{03} \quad (10)$$

$$\lambda v = h_{10}^r X + h_{11}^r Y + h_{13} \quad (11)$$

$$\lambda = h_{20}^r X + h_{21}^r Y + h_{23} \quad (12)$$

Now there are 3 equations and 3 unknowns. If Z^r was not known to be 0, there would be 3 equations and 4 unknowns (${}^r X, {}^r Y, Z^r, \lambda$), and the system would be underdetermined.

Substitute λ from (12) in to (10) to find ${}^r Y$ in terms of ${}^r X$:

$$(h_{20}^r X + h_{21}^r Y + h_{23})u = h_{00}^r X + h_{01}^r Y + h_{03} \quad (13)$$

$$(h_{20}u - h_{00})^r X + (h_{21}u - h_{01})^r Y = h_{03} - h_{33}u \quad (14)$$

$${}^r Y = \frac{h_{03} - h_{23}u - (h_{20}u - h_{00})^r X}{h_{21}u - h_{01}} \quad (15)$$

Substitute λ from (12) in to (11):

$$(h_{20}^r X + h_{21}^r Y + h_{23})v = h_{11}^r X + h_{12}^r Y + h_{13} \quad (16)$$

$$(h_{20}v - h_{10})^r X + (h_{21}v - h_{11})^r Y = h_{13} - h_{23}v \quad (17)$$

Substitute ${}^r Y$ from (15) in to (17):

$$(h_{20}v - h_{10})^r X + (h_{21}v - h_{11}) \left(\frac{h_{03} - h_{23}u - ((h_{20}u - h_{00})^r X)}{h_{21}u - h_{01}} \right) = h_{13} - h_{23}v \quad (18)$$

After simplification, this yields a solution for rX :

$${}^rX = \frac{h_{13} - h_{23}v - (h_{21}v - h_{11})\left(\frac{h_{03} - h_{23}u}{h_{21}u - h_{01}}\right)}{(h_{20}v - h_{10} - (h_{21}v - h_{11})\left(\frac{h_{20}u - h_{00}}{h_{21}u - h_{01}}\right))} \quad (19)$$

Substitute rX from (19) in to (15) to produce a closed-form solution for rY .